实验作业四　Vector RAG、GraphRAG 与 WikiRAG 对比实验

一、实验目的

1. 理解 RAG「先检索、后生成」的开卷考试思想与两条流水线（离线索引 / 在线检索生成），掌握分块与窗口预算的基本权衡（教材 8.1~8.3 节）。

2. 亲手实现 Vector RAG 基线：文本分块 → 嵌入向量化 → 余弦相似度 Top-K 检索 → 拼接生成（教材 8.2~8.4 节）。

3. 实现简化版 GraphRAG：三元组抽取 → networkx 建图 → 社区发现与摘要 → Local / Global 两类查询（教材 8.7 节）。

4. 实现 WikiRAG：把原始文档重写为「条目化知识库」，体会知识组织方式对检索质量的影响（教材 8.8 节）。

5. 用统一问题集完成三范式对比评测：Recall@5、MRR 手算与脚本互验、忠实度人工评分，回答「什么问题该用什么 RAG」（教材 8.10 节、表 8-4）。

二、实验环境

表 1　软件环境清单

项目	要求	说明
操作系统	Windows 10/11 或 macOS 均可	无 GPU 要求，全程 CPU 即可
Python	3.10 及以上	建议使用 TRAE 内置环境或 miniconda
嵌入模型	sentence-transformers：paraphrase-multilingual-MiniLM-L12-v2（默认）或 BAAI/bge-small-zh-v1.5	首次运行自动下载约 100MB；下载失败见「九、常见问题」Q1
LLM API（可选）	任一 OpenAI 兼容接口（DeepSeek / 智谱 / 通义 / Kimi 等）	仅生成与现场抽取环节需要；不配置也能完成全部检索评测
核心依赖	networkx、numpy、sentence-transformers、openai	一行命令安装，见下
提交物	实验报告（样板 docx）+ hw4_rag.py（含你的修改）	命名规则见「八、实验报告要求」

环境搭建命令（命令行 / TRAE 终端均可）：

```
pip install sentence-transformers networkx numpy openai
# 模型下载慢时，Windows PowerShell 先执行： $env:HF_ENDPOINT="https://hf-mirror.com"
python hw4_rag.py --mode eval    # 一键跑通三范式 × 15 题检索评测
```

三、TRAE IDE 使用要点

表 2 TRAE 核心功能与本实验的使用原则

功能	本实验的使用原则
Builder 模式	让 TRAE 按「五、实验任务」一次性生成 hw4_rag.py 项目骨架与三个检索类，再逐个调试；不要让它一步写完全部逻辑
Chat 模式	请它解释社区发现（Louvain / 贪心模块度）、RRF 等概念，追问到能向同学复述为止，并与教材 8.7 节对照
# 引用上下文	把教材第 8 章 PDF 拖入对话，让 TRAE 对照式（8-9）（8-10）实现指标计算，避免口径不一致
智能补全	三元组抽取 prompt、循环评测等重复代码交给补全；但 JSON 解析、实体匹配两处必须逐行自己检查
调试	检索「答非所问」时先打断点看召回的文档与相似度分数，再谈改参数——本实验的失败案例就藏在这里
终端	所有对比实验用命令行参数（--mode / --q / --scope）跑，保持代码不动，结果才可复现

四、原理回顾

4.1 三种 RAG 的知识组织差异（对照教材表 8-4）

同一份语料，三种范式只是「知识的整理方式」不同：Vector RAG 把文档切块后平铺进向量库；GraphRAG 先抽取实体三元组成图、再把稠密子图聚成社区并生成摘要；WikiRAG 则把散落的知识重写成一个个主题条目。检索时三者分别擅长回答三类不同的问题（表 3）。

表 3 三种 RAG 范式对比（对照教材表 8-4 精简）

维度	Vector RAG	GraphRAG	WikiRAG
知识组织	文档块平铺，语义向量索引	实体三元组成图 + 社区摘要	主题条目（标题 + 聚合正文）
检索方式	问题向量 → 余弦 Top-K	实体匹配沿边走 / 摘要匹配	条目向量 Top-K
擅长问题	局部事实（块内有完整答案）	多跳关系、全局归纳	主题聚焦的归纳问题
构建与更新成本	最低：切块入库即可	最高：抽取 + 建图 +	中等：重写条目，条

		摘要，更新要改图	目独立可增改

本实验的 15 道内置问题刻意分成三类（事实 / 多跳 / 全局），就是要让这三种组织方式的差异在指标上显形。

4.2 评测指标（必做手算，对照教材式 8-9 / 8-10）

为公平对比，三种范式的检索单位（文本块 / 邻域三元组 / 条目）统一映射回「文档」再计算指标：

Recall@5（式 8-9）＝ 命中的标准文档数 ÷ 标准文档总数（Top-5 内），衡量「该找的都找齐了吗」。

MRR（式 8-10）＝ 各题首个命中文档排名倒数的平均：MRR = (1/|Q|) Σ 1/rank，衡量「找得快不快、排得靠前吗」。

教材手算例：3 题首次命中排名 3、1、2 → MRR = (1/3 + 1/1 + 1/2) / 3 ≈ 0.611。报告中需另抽 3 题手算并与脚本输出核对。

忠实度（对照式 8-8 的 RAGAS 思想）：人工按 1~5 分给答案打分——5 分＝每句都能在检索到的资料里找到依据；1 分＝主要内容无依据（幻觉）。每种范式抽 3 题评分。

五、实验任务

任务一（Vector RAG 基线，必做）：读懂参考代码中 chunk_docs / Embedder / VectorRAG 三部分，运行 --mode eval 记录 Vector 列结果；观察多跳题（Q01~Q05）与事实题的表现差异，结合 Top-5 的窗口限制解释原因。

任务二（GraphRAG，必做）：检查 PRE_TRIPLES 是否覆盖多跳链的每一环（如 Q01 需要「李文瀚—任职于—人工智能学院」与「人工智能学院—院长—王启明」两环）；分别用 --scope local 与 --scope global 跑评测，对比两类查询在三类问题上的表现。

任务三（WikiRAG，必做）：阅读 PRE_ENTRIES 的 9 个条目——它们是「重写聚合」而非「复制」原文；运行评测，观察 Wiki 在全局类问题上的表现。

任务四（对比评测与手算，必做）：① 仿照表 4 自拟 5 道新题（每类至少 1 道）并标注 evidence；② 三范式 × 20 题跑评测，结果抄入报告表 3；③ 抽 3 题手算 MRR 并用 --mrr_check 复核；④ 每范式抽 3 题做忠实度评分；⑤ K 敏感性扫描——依次用 --k 1 / 2 / 3 / 5 重跑内置 15 题评测，把 Recall 填入报告表 4，观察哪类问题对检索窗口最敏感（教材 8.3 节窗口预算）。

任务五（进阶选做，二选一）：(a) 混合检索——给 Vector RAG 加一个字符 bigram 词面匹配分数，与向量分数按 RRF 融合（式 8-6），观察事实类问题是否提升；(b) 增量更新实验——在 CORPUS 中新增一篇 doc16（自拟内容），评估三种范式各需要改动什么、约花几分钟，写入报告。

表 4 内置问题集的三类设计（自拟题照此设计）

类型	题号	考察点	示例
局部事实 fact	Q06~Q10	块内有完整答案，检索器的基本盘	云山大学的校训是什么？
多跳关系 multi	Q01~Q05	跨文档关系链，GraphRAG 的主场	《大模型通识课》授课教师所在学院的行政负责人是谁？
全局归纳 global	Q11~Q15	答案散落多篇，考验知识组织方式	学校为学生提供哪些课程和课外活动？

六、实验步骤

步骤 1　创建项目：用 TRAE Builder 新建 hw4_rag 目录，按第五节任务生成骨架：语料 CORPUS、问题集 QUESTIONS、三个检索类、评测函数（可直接以第七节参考代码为底修改）。

步骤 2　跑通基线：python hw4_rag.py --mode eval。嵌入模型下载失败先按常见问题 Q1 处理；确有困难可用内置降级方案（字符 3-gram），但须在报告中注明。

步骤 3　逐题人工检查：用 --mode vector --q Q01 逐题打印检索到的文档，对照 evidence 看命中情况，把「检到了但排太后」与「根本没检到」两类失败各记 2 个例子。

步骤 4　GraphRAG 环节：先 --mode graph --q Q01 --scope local 观察沿边走的命中过程；再 --scope global 对比；把实体匹配的局限（同义说法如「云大」匹配不到「云山大学」）记入失败案例。

步骤 5　评测与手算：跑全量评测填报告表 3；用 --mrr_check 3,1,2 复核教材手算例；另抽 3 题手算 MRR 写进报告 4.5 节；再按任务四⑤做 K 敏感性扫描（--k 1 / 2 / 3 / 5）填报告表 4。

步骤 6　加题与选做：按任务四补 5 道自拟题重跑评测；有余力完成任务五 (a) 或 (b)。

七、参考代码（完整框架，单文件可直接运行）

以下 hw4_rag.py 与本指南配套：内置「云山大学」语料 15 篇、标注问题 15 道、预抽取三元组 40 条与条目库 9 条——不配 LLM API 也能完整跑通检索评测；配好 API 后用 --mode vector --q Q01 即可看到「检索 + 生成」完整链路。

注意：① 评测统一映射回文档级，三种检索单位不同的范式才可比；② 三元组与条目为预置版本，正式流程应由 LLM 抽取 / 改写（--mode extract 体验）；③ 新增题目务必同步标注 evidence，否则指标无意义。

```
# -*- coding: utf-8 -*-
"""
Vector RAG / GraphRAG / WikiRAG 对比实验 —— 实验作业四参考框架
对应教材第 8 章《检索增强生成》
单文件、开箱即跑：内置「云山大学」语料 15 篇、标注问题 15 道、
预抽取三元组 40 条与条目库 9 条。不配 LLM API 也能完成全部检索评测；
配置 API 后可用 --mode vector --q Q01 体验完整「检索 + 生成」。

常用命令：
  python hw4_rag.py --mode eval                  # 三范式 × 15 题检索评测（无需 API）
  python hw4_rag.py --mode vector --q Q01        # 单题：检索 + 生成
  python hw4_rag.py --mode graph --q Q01 --scope local
  python hw4_rag.py --mode wiki  --q Q11
  python hw4_rag.py --mode extract               # 调 LLM 现场抽三元组（需 API）
  python hw4_rag.py --mrr_check 3,1,2            # MRR 手算复核（对照式 8-10）
"""
import argparse, json, os, re, sys, time, zlib

# ============================================================
# 1. 语料库「云山大学」：15 篇虚构文档（三类问题的答案均已埋入）
# ============================================================
CORPUS = {
 "doc01": "云山大学位于岭南省云州市，建于 1958 年，是省属重点大学，现有全日制本科生约 2.1 万人。校训为「格物致知」。",
 "doc02": "人工智能学院成立于 2018 年，首任及现任院长为王启明教授。学院下设机器学习系、智能科学系、认知计算系三个系。",
 "doc03": "计算机学院前身为 1985 年成立的计算机系，现任院长陈国峰教授，设计算机科学与技术、软件工程两个本科专业。",
 "doc04": "人工智能学院教师：李文瀚，教授，2019 年入职，研究方向为检索增强生成，主讲《大模型通识课》；赵婉晴，副教授，研究方向为知识图谱，主讲《知识图谱导论》；孙浩然，讲师，研究方向为强化学习。",
 "doc05": "计算机学院教师：周天宇，教授，研究方向为分布式系统；林小雨，副教授，研究方向为数据库系统。",
 "doc06": "《大模型通识课》课程代码 AI2101，3 学分、32 学时，春季学期开设，授课教师李文瀚，面向全校本科生，无先修课程要求。",
 "doc07": "《知识图谱导论》课程代码 AI3305，2 学分，秋季学期开设，授课教师赵婉晴，面向人工智能学院研究生。",
 "doc08": "选课规则：本科生每学期最多修 30 学分；GPA 低于 2.0 给予学术警告。先修要求：《知识图谱导论》需先修《数据结构》，《大模型通识课》无先修要求。",
 "doc09": "奖学金体系：国家奖学金 8000 元/年，评定比例约 2%；校长奖学金 20000 元/年，全校每年 10 人；云山一等奖学金 3000 元/年，比例约 5%。",
```

```
 "doc10": "科研平台：认知计算全国重点实验室依托人工智能学院建设；岭南超算中心由云山大学与省科技厅共建，计算机学院参与运行管理。",
 "doc11": "学生社团：AI 协会，指导教师李文瀚，每周三晚组织论文研读；机器人战队，指导教师孙浩然，每年参加全国机器人大赛。",
 "doc12": "校历：春季学期 3 月 2 日开学、7 月 5 日放暑假；秋季学期 9 月 1 日开学、次年 1 月 15 日放寒假。",
 "doc13": "图书馆藏书 380 万册，开放时间为每天 7:00-22:00；人工智能分馆位于理科楼 B 座 3 层，收藏大模型与智能体专题图书。",
 "doc14": "校园交通：地铁 3 号线「云大站」距东门 200 米；校内校巴共 5 条线路，10 分钟一班。",
 "doc15": "国际交流：学校与 12 个国家的 47 所高校签有交换协议；人工智能学院与新加坡南洋理工大学有本科联合培养项目。",
}

# ============================================================
# 2. 问题集：3 类 × 5 题；evidence = 支持答案的文档（评测金标准）
#    fact=局部事实  multi=多跳关系  global=全局归纳
# ============================================================
QUESTIONS = [
 {"id": "Q01", "type": "multi",  "q": "《大模型通识课》授课教师所在学院的行政负责人是谁？",
  "evidence": ["doc04", "doc06", "doc02"]},
 {"id": "Q02", "type": "multi",  "q": "AI 协会的指导教师主讲课程的课程代码是什么？",
  "evidence": ["doc11", "doc04", "doc06"]},
 {"id": "Q03", "type": "multi",  "q": "认知计算全国重点实验室依托的学院成立于哪一年？",
  "evidence": ["doc10", "doc02"]},
 {"id": "Q04", "type": "multi",  "q": "《知识图谱导论》授课教师的研究方向是什么？",
  "evidence": ["doc07", "doc04"]},
 {"id": "Q05", "type": "multi",  "q": "机器人战队的指导教师的研究方向是什么？",
  "evidence": ["doc11", "doc04"]},
 {"id": "Q06", "type": "fact",   "q": "云山大学的校训是什么？", "evidence": ["doc01"]},
 {"id": "Q07", "type": "fact",   "q": "国家奖学金的金额是多少？", "evidence": ["doc09"]},
 {"id": "Q08", "type": "fact",   "q": "图书馆每天几点到几点开放？", "evidence":
["doc13"]},
 {"id": "Q09", "type": "fact",   "q": "人工智能学院成立于哪一年？", "evidence":
["doc02"]},
 {"id": "Q10", "type": "fact",   "q": "春季学期什么时候开学？", "evidence": ["doc12"]},
 {"id": "Q11", "type": "global", "q": "云山大学有哪些科研平台？分别依托谁建设？",
  "evidence": ["doc10", "doc02", "doc03"]},
 {"id": "Q12", "type": "global", "q": "学校的学生奖励体系包含哪些项目？", "evidence":
["doc09"]},
 {"id": "Q13", "type": "global", "q": "人工智能学院有哪些教师？各自做什么研究方向？",
  "evidence": ["doc04"]},
 {"id": "Q14", "type": "global", "q": "本科生选课有哪些限制和先修要求？", "evidence":
["doc08"]},
 {"id": "Q15", "type": "global", "q": "对想深入学习大模型的学生，学校提供哪些课程和课外活动？",
  "evidence": ["doc06", "doc11"]},
]

# ============================================================
# 3. 分块（教材 8.3 节：块长≈300 token，中文按 120 字近似，重叠 20 字）
# ============================================================
def chunk_docs(size=120, overlap=20):
    chunks, step = [], size - overlap
    for doc_id, text in CORPUS.items():
```

```
        i = 0
        while True:
            chunks.append((doc_id, text[i:i + size]))
            if i + size >= len(text):
                break
            i += step
    return chunks

# ============================================================
# 4. 嵌入后端：优先 sentence-transformers（教材 8.4 节）；
#    未安装/无网络时自动降级为字符 3-gram 相似度（流程可跑通，精度有限）
# ============================================================
class Embedder:
    def __init__(self, model_name):
        self.ok = False
        try:
            from sentence_transformers import SentenceTransformer
            self.model = SentenceTransformer(model_name)
            self.ok = True
            print(f"[embed] 已加载 {model_name}")
        except Exception as e:
            print(f"[embed] 加载失败（{type(e).__name__}），降级为字符 3-gram 相似度（精度
有限）")

    def encode(self, texts):
        import numpy as np
        if self.ok:
            return self.model.encode(list(texts), normalize_embeddings=True)
        M = np.zeros((len(texts), 512))
        for r, t in enumerate(texts):
            t = re.sub(r"\s", "", t)
            for i in range(max(1, len(t) - 2)):
                g = t[i:i + 3] if len(t) >= 3 else t
                M[r, zlib.crc32(g.encode("utf-8")) % 512] += 1
            n = np.linalg.norm(M[r]) or 1.0
            M[r] /= n
        return M



def _dedup(scored, k):
    docs, seen = [], set()
    for doc, s in scored:                    # scored 已按分数降序
        if doc not in seen:
            docs.append(doc)
            seen.add(doc)
            if len(docs) >= k:
                break
    return docs

# ============================================================
# 5. Vector RAG（教材 8.2~8.4 节）：块向量 → 余弦 Top-K → 文档序列
# ============================================================
class VectorRAG:
    name = "Vector RAG"

    def __init__(self, emb):
```

```
        self.emb = emb
        self.chunks = chunk_docs()
        self.M = emb.encode([c[1] for c in self.chunks])

    def retrieve(self, query, k=5, scope=None):
        qv = self.emb.encode([query])[0]
        scores = self.M @ qv
        order = scores.argsort()[::-1]
        hits = [(self.chunks[i][0], float(scores[i])) for i in order]
        return _dedup(hits, k)

# ============================================================
# 6. GraphRAG（教材 8.7 节简化版）
#    预抽取三元组 (头实体, 关系, 尾实体, 来源文档)；--mode extract 可现场抽取
# ============================================================
PRE_TRIPLES = [
 ("李文瀚", "任职于", "人工智能学院", "doc04"), ("李文瀚", "职称", "教授", "doc04"),
 ("李文瀚", "入职年份", "2019 年", "doc04"), ("李文瀚", "研究方向", "检索增强生成",
"doc04"),
 ("李文瀚", "主讲", "大模型通识课", "doc06"), ("李文瀚", "指导", "AI 协会", "doc11"),
 ("赵婉晴", "任职于", "人工智能学院", "doc04"), ("赵婉晴", "研究方向", "知识图谱", "doc04"),
 ("赵婉晴", "主讲", "知识图谱导论", "doc07"), ("孙浩然", "任职于", "人工智能学院", "doc04"),
 ("孙浩然", "研究方向", "强化学习", "doc04"), ("孙浩然", "指导", "机器人战队", "doc11"),
 ("人工智能学院", "院长", "王启明", "doc02"), ("人工智能学院", "成立于", "2018 年",
"doc02"),
 ("人工智能学院", "下设系", "机器学习系", "doc02"), ("人工智能学院", "下设系", "智能科学系",
"doc02"),
 ("人工智能学院", "下设系", "认知计算系", "doc02"), ("认知计算全国重点实验室", "依托", "人工
智能学院", "doc10"),
 ("岭南超算中心", "参与共建", "计算机学院", "doc10"), ("计算机学院", "院长", "陈国峰",
"doc03"),
 ("计算机学院", "前身", "计算机系", "doc03"), ("周天宇", "任职于", "计算机学院", "doc05"),
 ("周天宇", "研究方向", "分布式系统", "doc05"), ("林小雨", "任职于", "计算机学院", "doc05"),
 ("林小雨", "研究方向", "数据库系统", "doc05"), ("大模型通识课", "课程代码", "AI2101",
"doc06"),
 ("大模型通识课", "学分", "3 学分", "doc06"), ("大模型通识课", "开设学期", "春季学期",
"doc06"),
 ("知识图谱导论", "课程代码", "AI3305", "doc07"), ("知识图谱导论", "先修课程", "数据结构",
"doc08"),
 ("AI 协会", "指导教师", "李文瀚", "doc11"), ("机器人战队", "指导教师", "孙浩然", "doc11"),
 ("国家奖学金", "金额", "8000 元/年", "doc09"), ("校长奖学金", "金额", "20000 元/年",
"doc09"),
 ("云山一等奖学金", "金额", "3000 元/年", "doc09"), ("云山大学", "校训", "格物致知",
"doc01"),
 ("云山大学", "建校于", "1958 年", "doc01"), ("云山大学", "位于", "岭南省云州市", "doc01"),
 ("图书馆", "藏书量", "380 万册", "doc13"), ("春季学期", "开学日期", "3 月 2 日", "doc12"),
]

# 社区摘要（教材 8.7 节由 LLM 生成；此处预置 3 条，对应图的三个稠密子区）
COMMUNITY_SUMMARIES = [
 ("社区 C1 教学与课程：人工智能学院设三个系，开设《大模型通识课》（AI2101，李文瀚主讲）与"
  "《知识图谱导论》（AI3305，赵婉晴主讲），AI3305 需先修《数据结构》。",
  ["doc02", "doc04", "doc06", "doc07", "doc08"]),
```

```
 ("社区 C2 科研与国际：认知计算全国重点实验室依托人工智能学院，岭南超算中心由计算机学院参与共建；"
  "学院与新加坡南洋理工大学有联合培养项目。",
  ["doc02", "doc03", "doc10", "doc15"]),
 ("社区 C3 学生生活：奖学金分国家（8000 元）、校长（20000 元）、云山一等（3000 元）三级；"
  "社团有 AI 协会与机器人战队，图书馆藏书 380 万册、7:00-22:00 开放。",
  ["doc09", "doc11", "doc13"]),
]


class GraphRAG:
    name = "GraphRAG"

    def __init__(self, emb):
        import networkx as nx
        self.emb = emb
        self.G = nx.DiGraph()
        for h, r, t, src in PRE_TRIPLES:
            self.G.add_edge(h, t, rel=r, src=src)
        try:    # 教材 8.7 节用 Louvain；此处用 networkx 内置贪心模块度社区发现
            comms = nx.community.greedy_modularity_communities(self.G.to_undirected())
        except Exception:
            comms = [set(self.G.nodes)]
        print(f"[graph] 实体 {self.G.number_of_nodes()} 个 / 三元组 {len(PRE_TRIPLES)} 条 / 社区 {len(comms)} 个")
        self.summ_M = emb.encode([s for s, _ in COMMUNITY_SUMMARIES])

    def _entities(self, query, topn=2):
        ents = sorted([n for n in self.G.nodes if n in query], key=len, reverse=True)
        return ents[:topn]

    def retrieve(self, query, k=5, scope="local"):
        return self._local(query, k) if scope == "local" else self._global(query, k)

    def _local(self, query, k):
        import numpy as np
        ents = self._entities(query)
        seen, scored = set(), []
        frontier = ents[:]
        for _ in range(3):                          # 最多沿边走三跳
            nxt = []
            for e in frontier:
                edges = list(self.G.out_edges(e, data=True)) + list(self.G.in_edges(e, data=True))
                for h, t, d in edges:
                    key = (h, d["rel"], t)
                    if key not in seen:
                        seen.add(key)
                        scored.append((d["src"], f"{h} –{d['rel']}→ {t}"))
                        other = t if h == e else h
                        if other not in frontier:
                            nxt.append(other)
            nxt.sort(key=lambda x: 0 if self.G.out_degree(x) > 0 else 1)   # 实体优先展开
            frontier = nxt[:6]
```

```
        if not scored:                              # 未命中实体 → 退化为全局查询
            return self._global(query, k)
        qv = self.emb.encode([query])[0]
        sims = self.emb.encode([t for _, t in scored]) @ qv
        best = {}
        for (doc, txt), s in zip(scored, sims):
            if doc not in best or s > best[doc]:
                best[doc] = float(s)
        ranked = sorted(best.items(), key=lambda kv: -kv[1])
        return _dedup(ranked, k)

    def _global(self, query, k):
        qv = self.emb.encode([query])[0]
        scores = self.summ_M @ qv
        top = scores.argsort()[::-1][:2]            # 取最相关的 2 个社区
        docs = []
        for ci in top:
            for d in COMMUNITY_SUMMARIES[ci][1]:
                if d not in docs:
                    docs.append(d)
        return docs[:k]

# ============================================================
# 7. WikiRAG（教材 8.8 节）：条目化知识库，检索单位是「条目」
#    条目 = 标题 + 聚合正文（重写而非复制原文），映射回来源文档评测
# ============================================================
PRE_ENTRIES = [
 ("云山大学", "云山大学位于岭南省云州市，建于 1958 年，省属重点大学，本科生约 2.1 万人，校训「格物致知」。"
  "地铁 3 号线「云大站」距东门 200 米，校巴 5 条线路。校历：春季学期 3 月 2 日开学。",
  ["doc01", "doc12", "doc14"]),
 ("人工智能学院", "人工智能学院成立于 2018 年，院长王启明教授，下设机器学习系、智能科学系、认知计算系。"
  "认知计算全国重点实验室依托本院建设。与新加坡南洋理工大学有本科联合培养项目（每年 15 人）。",
  ["doc02", "doc10", "doc15"]),
 ("计算机学院", "计算机学院前身是 1985 年成立的计算机系，院长陈国峰教授，设计算机科学与技术、软件工程两个专业。"
  "参与岭南超算中心运行管理。",
  ["doc03", "doc10"]),
 ("人工智能学院教师", "李文瀚：教授，2019 年入职，研究检索增强生成，主讲《大模型通识课》，指导 AI 协会。"
  "赵婉晴：副教授，研究知识图谱，主讲《知识图谱导论》。孙浩然：讲师，研究强化学习，指导机器人战队。",
  ["doc04", "doc11"]),
 ("计算机学院教师", "周天宇：教授，研究分布式系统。林小雨：副教授，研究数据库系统。",
  ["doc05"]),
 ("大模型通识课", "《大模型通识课》（AI2101）：3 学分 32 学时，春季学期开设，李文瀚主讲，面向全校本科生，"
  "无先修要求，教材《人工智能——深度学习大模型智能体》。",
  ["doc06"]),
 ("知识图谱导论", "《知识图谱导论》（AI3305）：2 学分，秋季学期，赵婉晴主讲，面向人工智能学院研究生，"
  "需先修《数据结构》。",
  ["doc07", "doc08"]),
 ("奖学金体系", "国家奖学金 8000 元/年（约 2%）；校长奖学金 20000 元/年（全校 10 人）；"
```

```
  "云山一等奖学金 3000 元/年（约 5%）。",
  ["doc09"]),
 ("学生社团与校园生活", "AI 协会：指导教师李文瀚，每周三晚论文研读。机器人战队：指导教师孙浩然，"
  "参加全国机器人大赛。图书馆藏书 380 万册，每天 7:00-22:00 开放，人工智能分馆在理科楼 B 座。",
  ["doc11", "doc13"]),
]


class WikiRAG:
    name = "WikiRAG"

    def __init__(self, emb):
        self.emb = emb
        self.M = emb.encode([b for _, b, _ in PRE_ENTRIES])

    def retrieve(self, query, k=5, scope=None):
        qv = self.emb.encode([query])[0]
        scores = self.M @ qv
        order = scores.argsort()[::-1][:3]       # 条目级 Top-3
        docs = []
        for i in order:
            for d in PRE_ENTRIES[i][2]:
                if d not in docs:
                    docs.append(d)
        return docs[:k]

# ============================================================
# 8. 评测（教材式 8-9 / 8-10）：检索单位（块/邻域三元组/条目）统一映射回文档
# ============================================================
def evaluate(systems, k=5):
    print(f"\n===== 检索评测（文档级，Top-{k}） =====")
    print(f"{'范式':<12}{'问题类型':<10}{'Recall@' + str(k):>9}{'MRR':>8}   完整命中")
    for name, s in systems.items():
        for tp in ("fact", "multi", "global"):
            qs = [q for q in QUESTIONS if q["type"] == tp]
            recs, rrs, full = [], [], 0
            for q in qs:
                docs = s.retrieve(q["q"], k)
                ev = set(q["evidence"])
                got = ev & set(docs[:k])
                recs.append(len(got) / len(ev))
                rank = next((i + 1 for i, d in enumerate(docs) if d in ev), 0)
                rrs.append(1.0 / rank if rank else 0.0)
                full += (got == ev)

print(f"{name:<14}{tp:<12}{sum(recs)/len(recs):>7.3f}{sum(rrs)/len(rrs):>8.3f}
{full}/{len(qs)}")
    print("\n 提示：把本表抄进报告表 3；另抽 3 题手算 MRR，用 --mrr_check 复核（式 8-10）。")

# ============================================================
# 9. LLM 生成环节（可选）：任一 OpenAI 兼容接口
#    环境变量：LLM_API_BASE / LLM_API_KEY / LLM_MODEL
# ============================================================
```

```
EXTRACT_PROMPT = """从下面这篇大学介绍文档中抽取「头实体-关系-尾实体」三元组，只输出 JSON 数组，
不要输出任何其他文字，格式如 [["李文瀚","任职于","人工智能学院"]]。
关系用 2~4 字短语（任职于/研究方向/主讲/成立于 等），一篇文档抽 5~10 条。

文档：
{doc}"""

ANSWER_PROMPT = """你是问答助手。只依据下面资料回答问题；资料不足以回答时明确说「资料中未提及」，禁止编造。
问题：{q}
资料：
{ctx}
用 1~3 句话回答，句末用括号注明用到的资料编号。"""


def llm_chat(prompt, temperature=0.0):
    from openai import OpenAI
    base = os.environ.get("LLM_API_BASE")
    key = os.environ.get("LLM_API_KEY")
    model = os.environ.get("LLM_MODEL")
    if not (base and key and model):
        sys.exit("未配置 LLM：请设置环境变量 LLM_API_BASE / LLM_API_KEY / LLM_MODEL")
    client = OpenAI(base_url=base, api_key=key)
    resp = client.chat.completions.create(model=model, temperature=temperature,
                                          messages=[{"role": "user", "content":
prompt}])
    return resp.choices[0].message.content


def extract_triples():
    t0, out, bad = time.time(), [], 0
    for doc_id, text in CORPUS.items():
        raw = llm_chat(EXTRACT_PROMPT.replace("{doc}", text))
        try:
            arr = json.loads(raw[raw.index("["): raw.rindex("]") + 1])
        except Exception:
            bad += 1
            print(f"[warn] {doc_id} 解析失败：{raw[:60]}...")
            continue
        out += [(h, r, t, doc_id) for h, r, t in arr]
        print(f"{doc_id}: 抽到 {len(arr)} 条")
    print(f"\n共 {len(out)} 条 / 解析失败 {bad} 篇，耗时 {time.time() - t0:.0f}s。")
    print("请人工抽检后，把高质量三元组替换进脚本 PRE_TRIPLES。")

# ============================================================
# 10. 主入口
# ============================================================
def main():
    ap = argparse.ArgumentParser(description="实验作业四：三种 RAG 对比实验")
    ap.add_argument("--mode", choices=["eval", "vector", "graph", "wiki", "extract"],
default="eval")
    ap.add_argument("--q", default="Q01", help="问题编号，如 Q01")
    ap.add_argument("--scope", choices=["local", "global"], default="local")
    ap.add_argument("--k", type=int, default=5)
```

```
    ap.add_argument("--emb", default="paraphrase-multilingual-MiniLM-L12-v2")
    ap.add_argument("--mrr_check", default=None, help="如 3,1,2：打印 MRR 手算过程")
    args = ap.parse_args()

    if args.mrr_check:                          # 手算复核：MRR = (1/3+1/1+1/2)/3 ≈ 0.611
        ranks = [int(x) for x in args.mrr_check.split(",")]
        m = sum(1.0 / r for r in ranks) / len(ranks)
        print("MRR = (" + " + ".join(f"1/{r}" for r in ranks) + f") / {len(ranks)} =
{m:.3f}")
        return
    if args.mode == "extract":
        extract_triples()
        return

    emb = Embedder(args.emb)
    systems = {}
    if args.mode in ("eval", "vector"):
        systems["Vector RAG"] = VectorRAG(emb)
    if args.mode in ("eval", "graph"):
        systems["GraphRAG"] = GraphRAG(emb)
    if args.mode in ("eval", "wiki"):
        systems["WikiRAG"] = WikiRAG(emb)

    if args.mode == "eval":
        evaluate(systems, args.k)
        return

    name = {"vector": "Vector RAG", "graph": "GraphRAG", "wiki": "WikiRAG"}[args.mode]
    q = next(q for q in QUESTIONS if q["id"].upper() == args.q.upper())
    docs = systems[name].retrieve(q["q"], args.k, args.scope)
    print(f"\n[{q['id']}·{q['type']}] {q['q']}")
    print("检索到的文档：", docs)
    ctx = "\n".join(f"[{d}] {CORPUS[d]}" for d in docs)
    try:
        print("\n 回答：", llm_chat(ANSWER_PROMPT.replace("{q}", q["q"]).replace("{ctx}",
ctx)))
    except SystemExit:
        print("\n（未配置 LLM API，仅输出检索结果与上下文）\n" + ctx)


if __name__ == "__main__":
    main()
```

八、实验报告要求

报告以 Word / PDF 提交，命名「学号_姓名_实验作业四.docx」。请直接在配套的《实验报告样板》中填写，结构为：实验概述 → 三种 RAG 实现 → 对比实验设计 → 结果与数据记录 → 结果分析 → 失败案例 → AI 协作记录 → 思考题。检索命中截图、三元组 / 条目样例、指标表缺一不可。

表 5 评分标准（百分制）

项目	分值	评分要点
系统实现与跑通	30	三种范式均可运行；检索结果与 evidence 大体吻合；关键参数（块长 / Top-K / 社区数）有说明
对比评测与手算	25	20 题齐全（15 道内置 + 5 道自拟且标注合理）；表 3 / 表 4 数据与脚本输出一致；手算 MRR 过程完整；K 敏感性扫描覆盖 1 / 2 / 3 / 5 四档
结果分析	20	分问题类型解读三范式差异，结论与数据对应，能引用教材 8.7 / 8.8 节机理解释
失败案例分析	10	每范式至少 2 个真实案例，有现象、有原因归类（而非笼统说「效果不好」）
报告规范（含 AI 协作记录）	5	结构完整、图表有编号；如实记录与 TRAE 的协作过程
思考题	10	6 题均有依据作答，允许查资料但须注明来源

九、常见问题

Q1：嵌入模型下载不动？

先设镜像再跑：Windows PowerShell 执行 $env:HF_ENDPOINT="https://hf-mirror.com"；macOS / Linux 执行 export HF_ENDPOINT=https://hf-mirror.com。仍失败可用 --emb 指定本地已下载模型路径，或临时用内置降级方案（字符 3-gram，精度有限，报告须注明）。

Q2：没有 LLM API key 能完成作业吗？

能。检索评测（Recall@5 / MRR）全程不需要 API；生成与现场抽取环节可在报告说明用降级方案，或课后在提供 API 的环境补跑。

Q3：三元组抽取输出不是合法 JSON？

prompt 里已限定「只输出 JSON 数组」；参考代码对返回做了截取与解析，失败会打印原文并跳过。请在报告统计解析失败率——这本身就是 GraphRAG 的真实工程成本（8.7 节）。

Q4：图太碎 / 全连在一块？

抽取粒度按「一篇文档 5~10 条」控制；实体要归一（「云大」与「云山大学」合并为同一节点）。可在报告展示归一前后的图对比。

Q5：WikiRAG 的条目多长合适？

一个条目一个主题（人物 / 课程 / 机构），100~300 字为宜。条目过长会退回「块太大」的老问题，过短则聚合价值消失。

Q6：Vector RAG 多跳题得分很低（甚至全错）正常吗？

正常——单个文本块装不下跨文档关系链，桥接文档与问题的相似度低（思考题 (1) 追问机理）。但注意：嵌入模型语义泛化强时（如 bge 系列），桥接文档也可能被检回 Top-5 末位——此时用任务四⑤的 K 敏感性扫描把窗口收窄到 1~2，「检到但排太后」与「根本检不到」的差异会重新显现。

十、思考题（在报告样板中作答）

(1) 多跳问题的答案横跨多篇文档。以 Q01 为例写出完整的关系链（至少两条三元组），并解释：Vector RAG 为什么难以把链上的文档都排进 Top-5（提示：桥接文档与问题的词面 / 语义相似度低），GraphRAG 又是靠什么机制沿链取全的？（教材 8.7 节）

(2) 设某 3 题首次命中排名分别为 2、1、4，MRR 是多少？另 3 题 Recall@5 均为 1.0 而 MRR 仅 0.4，两个指标分别在衡量什么？为什么必须一起看？

(3) 全局问题（如 Q11「有哪些科研平台」）Vector RAG 会怎么失败？GraphRAG 的社区摘要为什么适合？WikiRAG 又靠什么机制解决？三者的本质差异是什么？

(4) 增量更新：学校新增一位教师。三种范式各自要改动什么（重算哪些块 / 图 / 条目）？从更新成本解释表 8-4 中三种范式的工程定位。

(5) 忠实度人工评分的主观性如何缓解？RAGAS 用 LLM 自动评忠实度的思路是什么（式 8-8），这种「用 LLM 评 LLM」又引入了什么新偏差？

(6) 回顾你与 TRAE 协作实现 GraphRAG 的过程：抽取 prompt 迭代了几版、哪次改动最关键？AI 生成的代码里你必须亲自检查哪些地方（JSON 解析、实体归一化、多跳展开的边界）？